

Hybrid NPU + GPU Execution

Running LLM prompt processing on the NPU and token generation on the GPU, inside a single ONNX Runtime session.

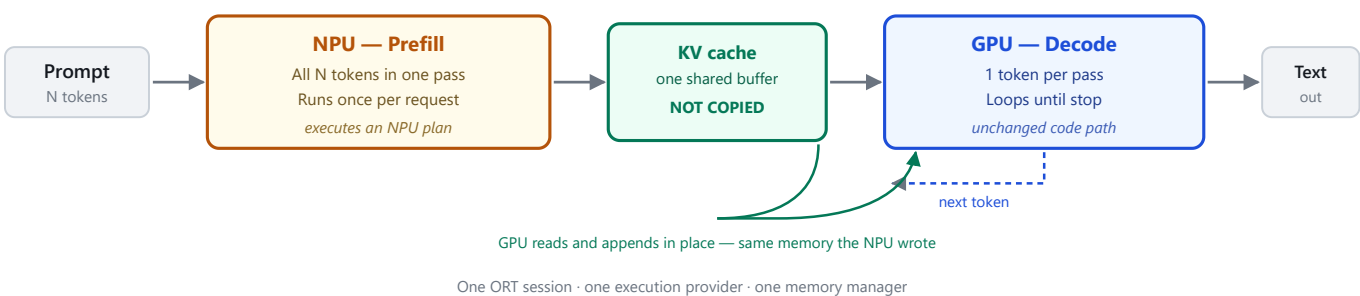
hip-ep · MorphiZen HIP Execution Provider · design overview

The idea

Generating text has two phases with **opposite** hardware appetites. Today both run on the GPU. This project sends each phase to the processor that suits it, without leaving the ORT session.

PHASE	WHAT IT DOES	BOTTLENECK	BEST HARDWARE
Prefill TTFT	Reads the whole prompt at once	Compute — large dense matrix work	NPU — dense systolic arrays
Decode TPS	Emits one token at a time, in a loop	Memory bandwidth — re-reads weights per token	GPU — high bandwidth

The prompt is processed once; a token is generated hundreds of times. Speeding up the first improves responsiveness, and the second sets sustained speed — so the split targets both at once.



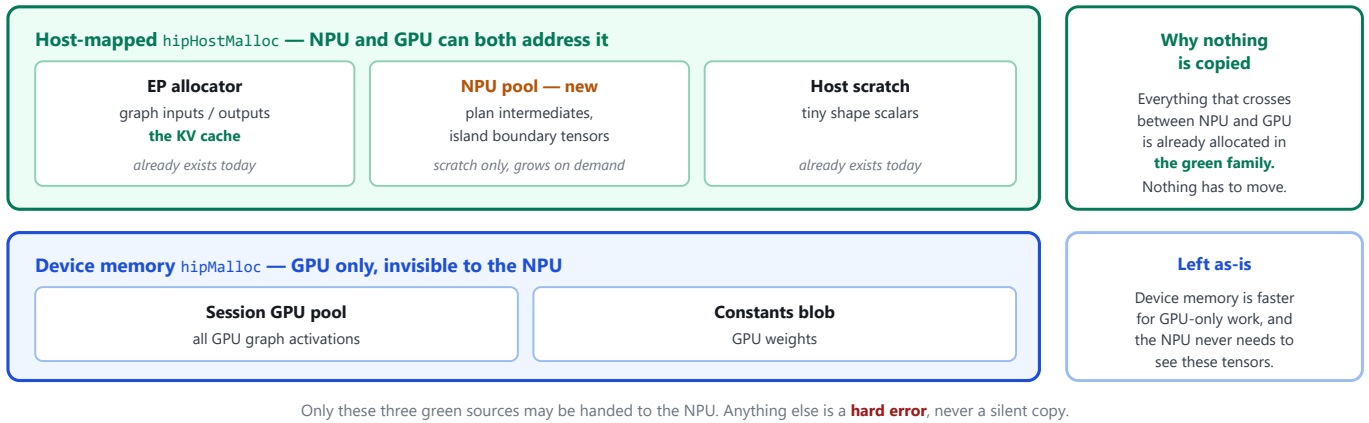
Runtime flow. The handoff between processors is the KV cache. Because both sides address the same buffer, the switch costs nothing — no tensor is copied, staged or repacked.

Zero copy is a requirement, not an optimization.

The KV cache is the largest tensor in the system. If it had to be copied whenever execution changed processor, the copy would cost more than the NPU saves and the idea collapses. So it is enforced rather than hoped for: counters assert zero copies, and a buffer is accepted only because of *which allocator produced it* — never because reading it happens to work.

Where tensors live

Zero copy is really a **memory** question, so this is the part worth understanding. There is one memory *manager*, but not one pool — because whether the NPU can reach a buffer is a physical property of how it was allocated, not a policy choice.



The memory model. Five classes, split by whether the pages are host-accessible — that is the only property that decides whether the NPU can address them.

Why not one pool for both?

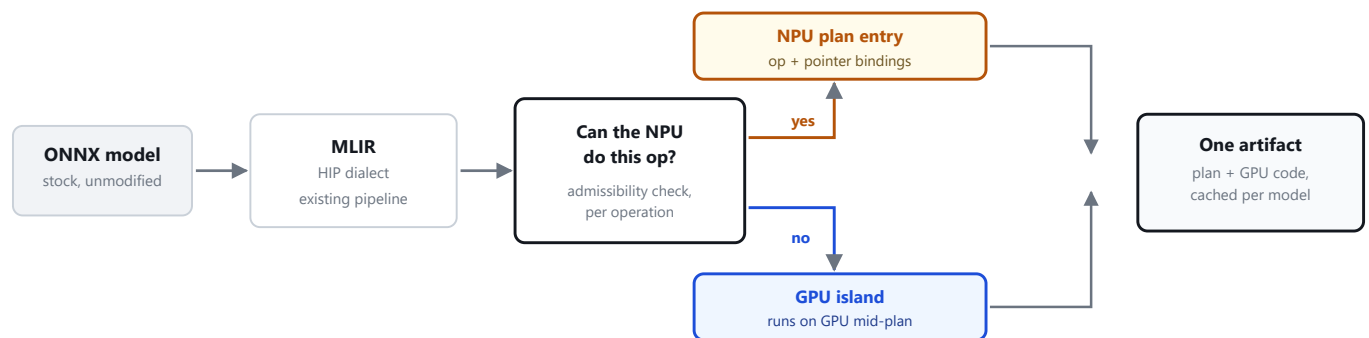
The GPU pool is device memory *by design*, and the graph memory planner packs every transient into it. Making it NPU-visible would mean moving all GPU activations to slower host-mapped memory and reworking the planner — for one execution target. The NPU pool is small and additive instead.

So what is the "NPU pool" for?

Only the plan's own scratch: intermediates and island boundary tensors. It is **not** where the KV cache lives — that already comes from the EP allocator, which is already host-mapped. The new pool mirrors the host-scratch pool that exists today.

How a model gets there

Nothing about the model changes. The same ONNX file compiles to **one artifact containing two things**: a plan the NPU follows, and GPU code as before.



An operation the NPU cannot do correctly is **rejected, never silently approximated** — it falls back to the GPU instead.

Compilation flow. The plan is **data, not code** — an ordered list of operations with pointer bindings. That keeps it inspectable, cacheable and testable without hardware.

Why "reject, never approximate"

Our operations carry optional features the NPU library lacks. Quietly ignoring one computes a **different function** and still returns plausible-looking numbers. Declining is strictly safer, so an unrecognized attribute means fall back to the GPU.

Why some ops stay on the GPU

The NPU has no kernel for a few operations, notably the embedding lookup, and none for appending to the KV cache. Those run as **islands** inside the plan. Expected, and they scale with model depth rather than op count.

What is being built

PIECE	PURPOSE	WHY IT EXISTS
Shim library	Thin C interface between the EP and the NPU driver stack	Isolates conflicting dependencies; lets a mock stand in with no hardware
NPU pool	Host-mapped scratch for plan intermediates	Keeps plan buffers NPU-visible without touching the GPU pool
Buffer registry	Records which allocator produced each pointer	Makes a violation a hard error instead of undefined behaviour
Copy counters	Count every boundary copy at run time	Makes zero copy testable ; a copy yields correct numbers, so numerics can't detect it
Plan emitter	Turns claimed MLIR into a plan, or declines with a reason	Compiler side, verifiable without an NPU
Plan interpreter	Walks the plan and dispatches each entry	Runtime side; restartable, so falling back mid-plan is always safe

Roadmap

39 tasks in nine phases, ordered so each phase makes the next one testable.

PHASE	NAME	GOAL
0	Spikes	Prove the risky assumptions before building on them
1	Shim and C ABI	The boundary to the NPU stack, plus the mock behind it
2	NPU memory class	Pool, registry and copy counters — zero-copy enforcement
3	Plan emission	Compiler emits a plan, or declines with a reason
4	Dispatch and interpretation	Choose prefill vs decode; execute the plan
5	Operator coverage	Map the operators; attention is the large one
6	GPU islands	Let unmapped operations run mid-plan on the GPU
7	Weight streaming	Get weights into NPU-visible memory once, at load
8	End to end	Full generation, both models, copy counters at zero

Overview only. Full detail in `docs/design/hybrid-npu-gpu-design.md` (architecture and decisions) and `docs/design/hybrid-npu-gpu-tasks.md` (task breakdown).